Anchor-Guided Dequantization
for Federated Learning

Repairing low-bit client updates with full-precision anchors

Jacob Simmons (jss0112@auburn.edu)
Owen Strength (ods0005@auburn.edu)
Auburn University
COMP 5970/6970 Course Project

CIFAR-10 10 clients INT2 / INT4 / INT8 server-side CorrectionNet

Problem Setting

Standard FL assumption

FedAvg works well when uploaded client updates are already comparable and can be averaged directly.

What changes here

Clients are **bit-width heterogeneous**.

- Anchor clients upload **FP32** deltas.
- Constrained clients upload **INT8 / INT4 / INT2** deltas.
- The server averages *clean* and *biased* updates together.

Failure mode

Quantization error is not just random noise.

- Low-bit updates lose magnitude information.
- With very few levels, signs cancel and the average collapses.
- The global model absorbs that distortion every round.

Bias at the server

If the constrained upload is a distorted version of the full-precision update, then averaging clean and quantized deltas pushes the server in the wrong direction.

Why Existing Fixes Are Not Enough

FedAvg

- No dequantization step at all.
- Relies on client averaging to cancel quantization noise.
- That assumption breaks hardest at INT2.

Simplified ProWD baseline

- Train a 3-layer residual MLP on the *current round only*.
- Pull every constrained client toward the *mean* anchor delta.
- No persistent replayed supervision across rounds.

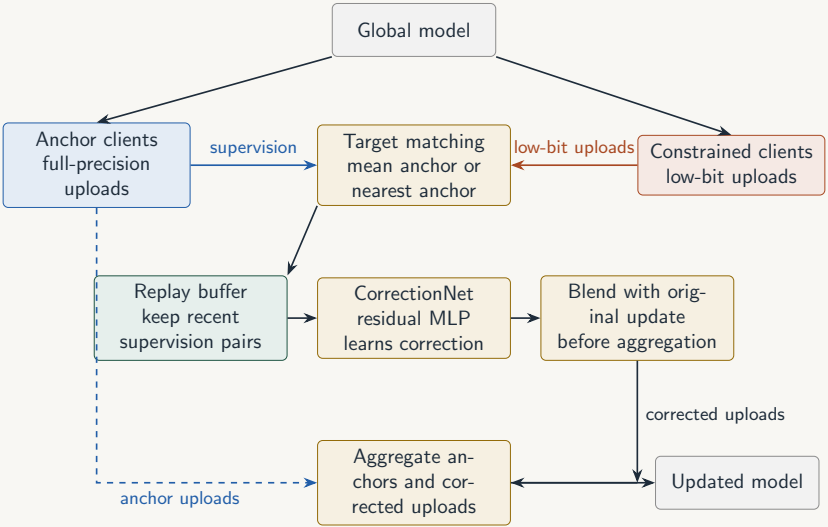
What AGD-Smart changes

- Use full-precision clients as **direct supervision**.
- Keep that supervision in a **replay buffer**.
- Match each constrained client to a **relevant anchor target**.
- Blend correction with the original delta when non-IID structure matters.

Core idea

Learn a server-side corrector that turns a low-bit upload into a more full-precision-like upload using anchors as teachers.

Method Overview



What Happens Each Round

1. **Collect two kinds of updates**
   - Anchors send clean FP32 deltas Δw_a^{fp}.
   - Constrained clients send quantized deltas Δw_j^q.
2. **Build targets for constrained clients**
   - Baseline mode: target the *mean* anchor delta.
   - Smart mode: choose the anchor whose update points in the most similar direction.
3. **Train on buffered supervision**
   - Store $(\Delta w_j^q, \Delta w_j^{\mathrm{target}})$ pairs.
   - Keep the most recent **10 rounds**.
   - Normalize buffered inputs with a scalar z-score before training.
4. **Correct, blend, aggregate**
   - Apply the learned map to each constrained delta.
   - Blend corrected and original directions when needed.
   - Aggregate anchors plus corrected constrained updates.

CorrectionNet and Training Objective

CorrectionNet

- One encoder layer
- Three residual blocks
- Zero-initialized decoder

Zero-init matters: training starts from an identity map, so the network learns *bias correction* instead of a full rewrite of the update.

Loss

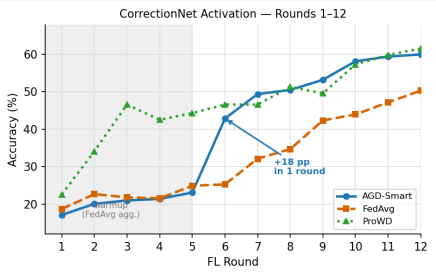
- Reconstruction term: make the corrected upload look like the chosen anchor target.
- Optional distillation term: make the repaired update produce similar logits on a server validation split.
- $\tau = 4$ in the reported settings.

Why Warmup Matters

Warmup

For the first **5 rounds**, the server trains CorrectionNet but does not apply correction yet.

- The replay buffer fills with real supervision pairs.
- The untrained network cannot damage early optimization.
- Correction activates only after there is enough signal to use.



Interpretation: the jump appears exactly when correction turns on after warmup.

Why Blending Matters

Blended correction

Use **80% corrected update + 20% original update** in the smart configuration.

In the smart configuration, $\beta = 0.8$ keeps 20% of the original client direction.

Why not use full correction all the time?

- Under non-IID data, different clients can have legitimately different update directions.
- Pure correction can over-homogenize those updates toward anchor behavior.
- Blending keeps some client-specific signal while still removing low-bit bias.

Experimental Setup

Data and model

- CIFAR-10
- 10 total clients, full participation
- Shared CNN backbone
- 2 local epochs per round

Baselines

- FedAvg
- Simplified ProWD baseline

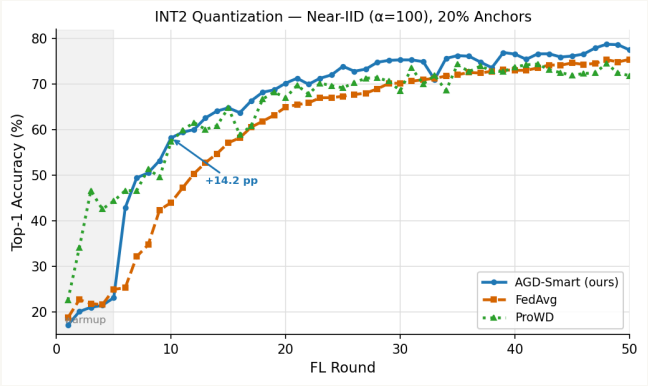
Reported regimes

Setting	Bits	Anchors	Rounds
Near-IID	INT8	5/10	50
Near-IID	INT4	5/10	50
Near-IID	INT2	2/10	50
Non-IID	INT8	5/10	50

Question

When does learned dequantization help enough to justify the extra server-side machinery?

Main Result: INT2 Is Where the Method Helps



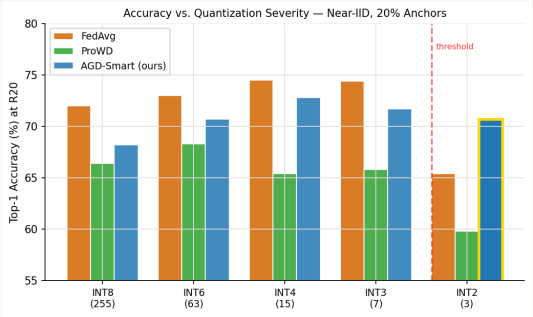
Setting

Near-IID CIFAR-10
INT2 quantization
20% anchors
50 rounds

Headline numbers

- R10: **58.2** vs 44.0 (FedAvg) and 57.3 (ProWD)
- R50: **77.5** vs 75.4 (FedAvg) and 71.8 (ProWD)
- Gain: **+14.2** at R10 and **+2.1** at R50 over FedAvg

Why INT2 Is the Threshold



Key observation

- AGD-Smart only beats FedAvg once quantization becomes too coarse for averaging to recover the signal.

Why INT2 is different

- Only **3 effective levels**: negative, zero, positive.
- Large and small gradients collapse to the same code.
- Opposite signs cancel across clients.

Interpretation

- At INT3 and above, FedAvg usually has enough resolution to survive.
- At INT2, the learned correction becomes genuinely useful.

Other Regimes

Near-IID, INT8

AGD-Smart reaches **81.2** at R50, beats ProWD by **+1.9**, and is slightly behind FedAvg at the end.

Near-IID, INT4

AGD-Smart reaches **82.5** at R50, beats ProWD by **+4.8**, and nearly matches FedAvg.

Non-IID, INT8

AGD-Smart reaches **75.3** at R50 and still beats ProWD by **+2.5**, but FedAvg benefits from preserving client diversity.

Across all reported settings, the stable pattern is: **AGD-Smart** $>$ **ProWD**, while **AGD-Smart** $>$ **FedAvg** only appears in the extreme INT2 regime.

Takeaways

What the method is doing

It uses clean anchor-client updates to supervise a persistent server-side corrector for low-bit client deltas.

Why it beats ProWD

- replayed supervision instead of round-by-round forgetting,
- client-specific smart matching instead of a single mean target,
- warmup and blending so correction turns on more safely.

Main conclusion

This is not a universal replacement for FedAvg. It is a targeted fix for the regime where quantization is severe enough that averaging alone fails, especially INT2.

Selected References

- McMahan et al. (2017), *Communication-Efficient Learning of Deep Networks from Decentralized Data*.
- Li et al. (2020), *Federated Optimization in Heterogeneous Networks*.
- Karimireddy et al. (2020), *SCAFFOLD: Stochastic Controlled Averaging for Federated Learning*.
- Wang et al. (2020), *Tackling the Objective Inconsistency Problem in Heterogeneous Federated Optimization*.
- Reisizadeh et al. (2020), *FedPAQ: A Communication-Efficient Federated Learning Method with Periodic Averaging and Quantization*.
- Yoon et al. (2022), *Bitwidth Heterogeneous Federated Learning with Progressive Weight Dequantization*.
- Seo et al. (2024), *FedShift: Robust Federated Learning Aggregation Scheme in Resource Constrained Environment via Weight Shifting*.
- Chen and Vikalo (2024), *Mixed-Precision Quantization for Federated Learning on Resource-Constrained Heterogeneous Devices*.